```python
from fastapi import FastAPI, HTTPException, Query, status
from pydantic import BaseModel, Field
from typing import List, Optional
from uuid import uuid4

app = FastAPI(
    title="Book CRUD API",
    description="Datheon Python + FastAPI Backend Development Internship - Task 1",
    version="1.0.0",
)

class BookBase(BaseModel):
    title: str = Field(..., min_length=1, max_length=200)
    author: str = Field(..., min_length=1, max_length=100)
    year: Optional[int] = Field(None, ge=0, le=2100)
    genre: Optional[str] = Field(None, max_length=50)

class BookCreate(BookBase):
    pass

class BookUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1, max_length=200)
    author: Optional[str] = Field(None, min_length=1, max_length=100)
    year: Optional[int] = Field(None, ge=0, le=2100)
    genre: Optional[str] = Field(None, max_length=50)

class Book(BookBase):
    id: str

# In-memory database
books_db = {}

@app.get("/", tags=["Root"])
def root():
    return {"message": "Book CRUD API", "docs": "/docs"}

@app.get("/books", response_model=List[Book], tags=["Books"])
def get_books(
    title: Optional[str] = Query(None, description="Filter by title (case-insensitive contains)"),
    author: Optional[str] = Query(None, description="Filter by author (case-insensitive contains)"),
    genre: Optional[str] = Query(None, description="Filter by genre (case-insensitive contains)"),
    year: Optional[int] = Query(None, description="Filter by exact year"),
):
    results = list(books_db.values())
    if title:
        results = [b for b in results if title.lower() in b["title"].lower()]
    if author:
        results = [b for b in results if author.lower() in b["author"].lower()]
    if genre:
        results = [b for b in results if b.get("genre") and genre.lower() in b["genre"].lower()]
    if year is not None:
        results = [b for b in results if b.get("year") == year]
    return results

@app.get("/books/{book_id}", response_model=Book, tags=["Books"])
def get_book(book_id: str):
    book = books_db.get(book_id)
    if not book:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Book not found")
    return book

@app.post("/books", response_model=Book, status_code=status.HTTP_201_CREATED, tags=["Books"])
def create_book(book: BookCreate):
    book_id = str(uuid4())
    book_dict = book.model_dump()
    book_dict["id"] = book_id
    books_db[book_id] = book_dict
    return book_dict

@app.put("/books/{book_id}", response_model=Book, tags=["Books"])
def update_book(book_id: str, book_update: BookUpdate):
    book = books_db.get(book_id)
    if not book:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Book not found")
    update_data = book_update.model_dump(exclude_unset=True)
    updated_book = {**book, **update_data}
    books_db[book_id] = updated_book
    return updated_book

@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["Books"])
def delete_book(book_id: str):
    if book_id not in books_db:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Book not found")
    del books_db[book_id]
    return None
```